

CARGO MATE
SMART ROBOTIC INVENTORY SYSTEM WITH CLOUD
INTEGRATION

25-26J-032

Design Document (DD) of Robotic Arm

B. Kethmi Senara

B.Sc. (Hons) Degree in Information Technology specialized in
Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

January 9, 2026

Table of Contents

Table of Contents	ii
1. Introduction	1
1.1 Purpose	1
1.2 Scope.....	1
1.3 Definitions, Acronyms, and Abbreviations.....	2
1.4 Overview	2
2. Overall Description	3
2.1 Product Perspective.....	3
2.1.1 System Interfaces	3
2.1.2 User Interfaces.....	3
2.1.3 Hardware Interfaces	4
2.1.4 Software Interfaces	4
2.1.5 Communication Interfaces	4
2.1.6 Memory Constraints	4
2.1.7 Operations.....	4
2.1.8 Site Adaptation Requirements.....	4
2.2 Product Functions	5
2.3 User Characteristics	5
2.4 Constraints	5
2.5 Assumptions and Dependencies	5
2.6 Apportioning of Requirements	5
3. Specific Requirements	6
3.1 External Interface Requirements.....	6

3.1.1 User Interfaces	6
3.1.2 Hardware Interfaces	6
3.1.3 Software Interfaces	6
3.1.4 Communication Interfaces	6
3.2 Architectural Design	7
3.2.1 High-Level Architectural Design	7
3.2.2. Process flow diagrams	8
3.2.3. Circuit Diagram	9
3.2.4. Hardware and Software Requirements with Justification	9
3.2.5. Risk Mitigation Plan with Alternative Solution Identification	10
3.2.6. Cost Benefit Analysis	10
3.3 Performance Requirements.....	10
3.4 Design Constraints	10
3.5 Attributes of Software Systems	11
3.5.1 Dependability	11
3.5.2 System Dependability	11
3.5.3 Security	11
3.5.4 Maintainability	12
3.6 Other Requirements	12
4. References	13
5. Supporting Information	13
5.1 Table of Contents and Index.....	13
5.2 Appendices	13
Appendix A - Test Cases	13

Appendix B - Pin Mapping Table	14
---	-----------

List of figures

Figure 1 - High level Architecture diagram.....	7
Figure 2 - Storage Process Flow	8
Figure 3 - Retrieval Process Flow	8
Figure 4 - Circuit Diagram.....	9

1. Introduction

1.1 Purpose

This design document details the design and implementation of a robotic arm system based on the ESP32 platform. The document presents a complete system architecture, as well as the integration of hardware and software components, interface specifications, performance requirements and all known limitations and constraints for this project. It describes how the various components interact to provide the desired functionality of a robotic arm.

This document will serve as a technical roadmap for the development of this system. It outlines all of the necessary steps to be taken during development, as well as to test and validate the final product. This document also includes information necessary for the future maintenance and repair of the robotic arm system, as well as to allow for any enhancements/changes that may become necessary in the future.

1.2 Scope

This Paper provides a step-by-step guide for designing the entire robotic arm embedded system including:

- Mechanical actuator designed with servo motors
- Sensor Integration that detects objects and controls gripper force
- Embedded Firmware development using ESP32 (Microcontroller)
- ECU (Electronic Control Unit) and ECU + Embedded firmware communication Interfaces
- Performance, Reliability and Maintainability (RRM) considerations

The Paper is designed for functional and Logical design only, and does NOT include Manufacturing-Level mechanical drawings

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
ESP32	Low-power microcontroller with integrated Wi-Fi and Bluetooth
IR Sensor	Infrared sensor used for object detection
FSR	Force Sensitive Resistor used to measure gripping force
PWM	Pulse Width Modulation used for servo control
GPIO	General Purpose Input/Output pins
ADC	Analog-to-Digital Converter
DD	Design Document

1.4 Overview

This robotic arm system was originally an inexpensive design intended to perform pick-and-place functions. The movement of the robotic arm is accomplished through the use of servo motors, with the controls being managed via an ESP32 microcontroller. An infrared (IR) sensor is used to locate objects, and the amount of gripping pressure applied is determined by the FSR sensor. Intended for use in education, lab automation and small industrial demonstrations, this solution provides an effective means of automating manual tasks that would require significant time and effort otherwise.

2. Overall Description

The robotic arm system implements embedded pieces of hardware alongside real time software/firmware to provide accurate and repeatable manipulation/handling of items. The hardware pieces are comprised of servos, sensors and an ESP32 Microcontroller. The real-time Firmware synchronizes the process/tooling of the Motors and Handles with the Sensor data and vice versa in real-time. The system's design is primarily focused on using low-cost components that are widely available for education/prototype applications. The Modular Design permits changes/upgrades to individual items (Sensor, Actuator, etc.) without altering the complete system. The system has also been designed to scale up with future improvements (i.e. Wireless Control, Additional Sensors) while maintaining Reliable and Stable Performance during continuous use.

2.1 Product Perspective

This system can be used independently of industrial robotic controllers and provides a completely autonomous embedded solution. Compared to traditional robotic arms, the system will have lower costs and simpler control logic while still providing flexibility to customize the design. Compact size makes it possible to be used for educational/prototype applications.

2.1.1 System Interfaces

- ESP32 firmware interface via Arduino IDE
- Used for code development, uploading, and debugging

2.1.2 User Interfaces

- Serial Monitor displaying system states, sensor values, and error messages
- Status indications through serial logs for debugging and testing

2.1.3 Hardware Interfaces

- Servo motors connected via PWM-capable GPIO pins for precise movement
- IR sensor connected to digital input pins for object detection
- FSR sensor connected to ESP32 ADC pins for force measurement
- External regulated power supply for motors to ensure stable operation

2.1.4 Software Interfaces

- Arduino IDE and ESP32 board support package
- Embedded C firmware libraries for servo and sensor handling

2.1.5 Communication Interfaces

- USB serial communication for monitoring and control
- Wi-Fi module reserved for future remote monitoring

2.1.6 Memory Constraints

- ESP32 internal RAM and flash memory limitations
- Optimized memory usage for real-time control

2.1.7 Operations

- System startup and initialization
- Servo and sensor calibration
- Object detection and validation
- Pick and place operations

2.1.8 Site Adaptation Requirements

- Stable power supply
- Proper lighting for IR sensor accuracy
- Secure mounting surface for the robotic arm

2.2 Product Functions

- Detect objects using IR sensors
- Perform multi-axis movement using servo motors
- Control gripping force using FSR feedback
- Execute automated pick-and-place tasks
- Provide feedback through serial interface

2.3 User Characteristics

- Undergraduate engineering students
- Beginners in robotics and embedded systems
- Users with basic electronics and programming knowledge

2.4 Constraints

- Limited project budget
- Servo motor torque and speed limitations
- Embedded system processing and memory constraints

2.5 Assumptions and Dependencies

- Objects remain within predefined size and weight limits
- Servo motors operate within rated voltage levels
- ESP32-compatible development tools are available

2.6 Apportioning of Requirements

- Phase 1: Basic robotic arm movement (completed)
- Phase 2: Object detection and pick-and-place integration (completed)
- Phase 3: Grip force optimization using FSR (ongoing)

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- Serial monitor interface for displaying system status and operational messages
- Real-time output of sensor readings (IR sensor and FSR values)
- Indication of pick-and-place stages and servo movement states
- Debug and error messages to support testing, calibration, and troubleshooting

3.1.2 Hardware Interfaces

- PWM-capable GPIO pins used to control servo motor angles
- Digital input pins connected to IR sensors for object detection
- Analog input pins connected to FSR sensors for grip force measurement
- External power interface for supplying adequate power to servo motors

3.1.3 Software Interfaces

- Arduino IDE used for firmware development and deployment
- Servo control libraries for PWM-based motor actuation
- ESP32 hardware abstraction libraries for GPIO, ADC, and timing control
- Sensor libraries for reliable data acquisition and processing

3.1.4 Communication Interfaces

- USB serial interface for programming, debugging, and monitoring
- Serial communication protocol for data exchange with host system
- Built-in Wi-Fi interface reserved for future wireless control or monitoring

3.2 Architectural Design

3.2.1 High-Level Architectural Design

The system follows a modular embedded architecture consisting of sensor modules, motor control modules, and a central control unit implemented on the ESP32. Each module operates independently but communicates through well-defined interfaces.

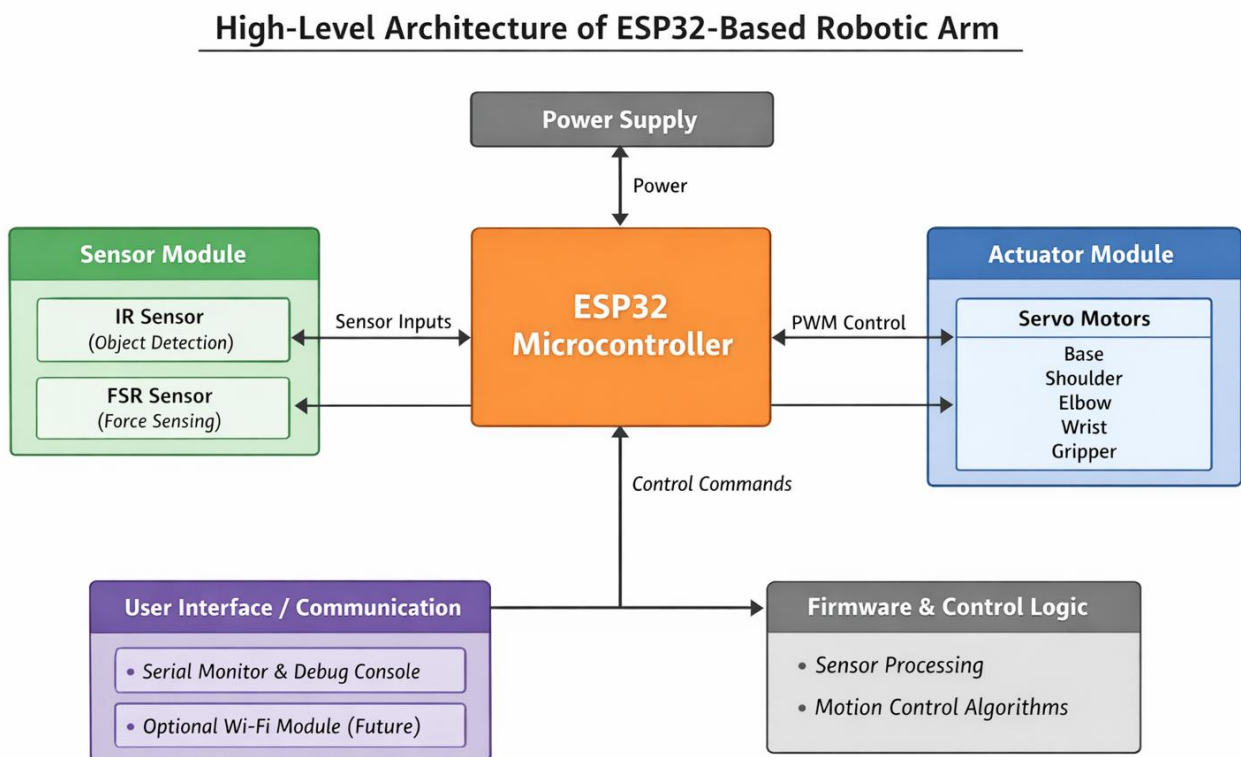


Figure 1 - High level Architecture diagram

3.2.2. Process flow diagrams

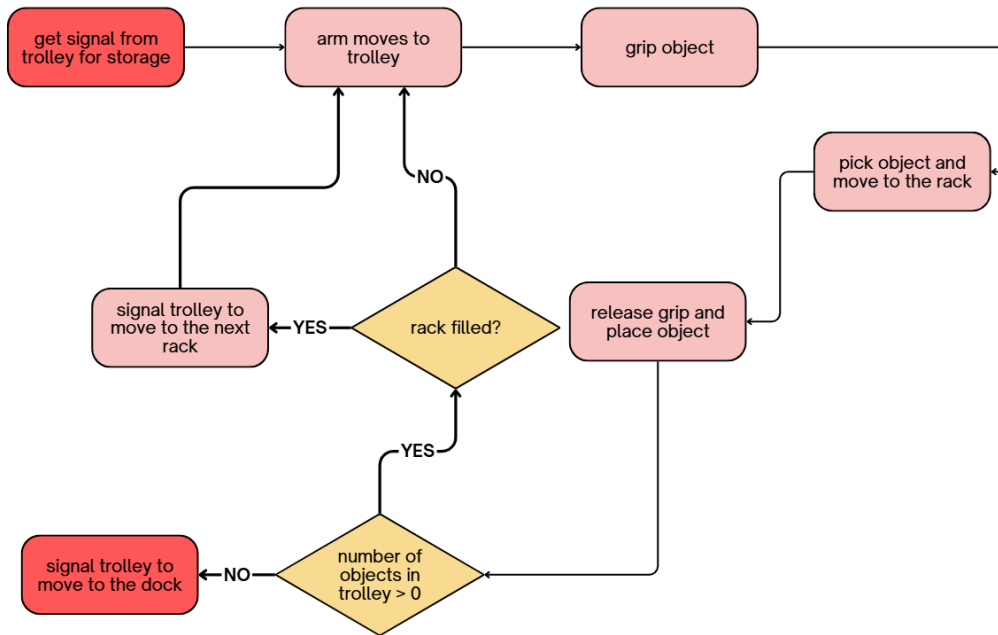


Figure 2 - Storage Process Flow

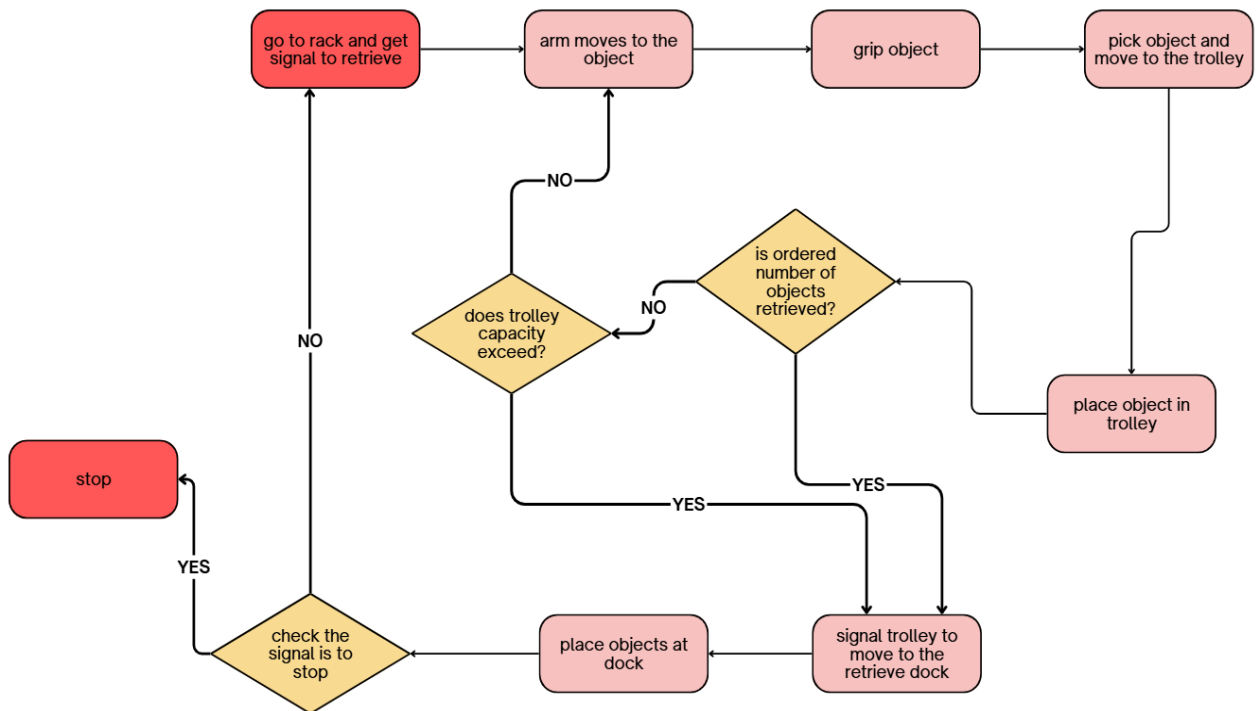


Figure 3 - Retrieval Process Flow

3.2.3. Circuit Diagram

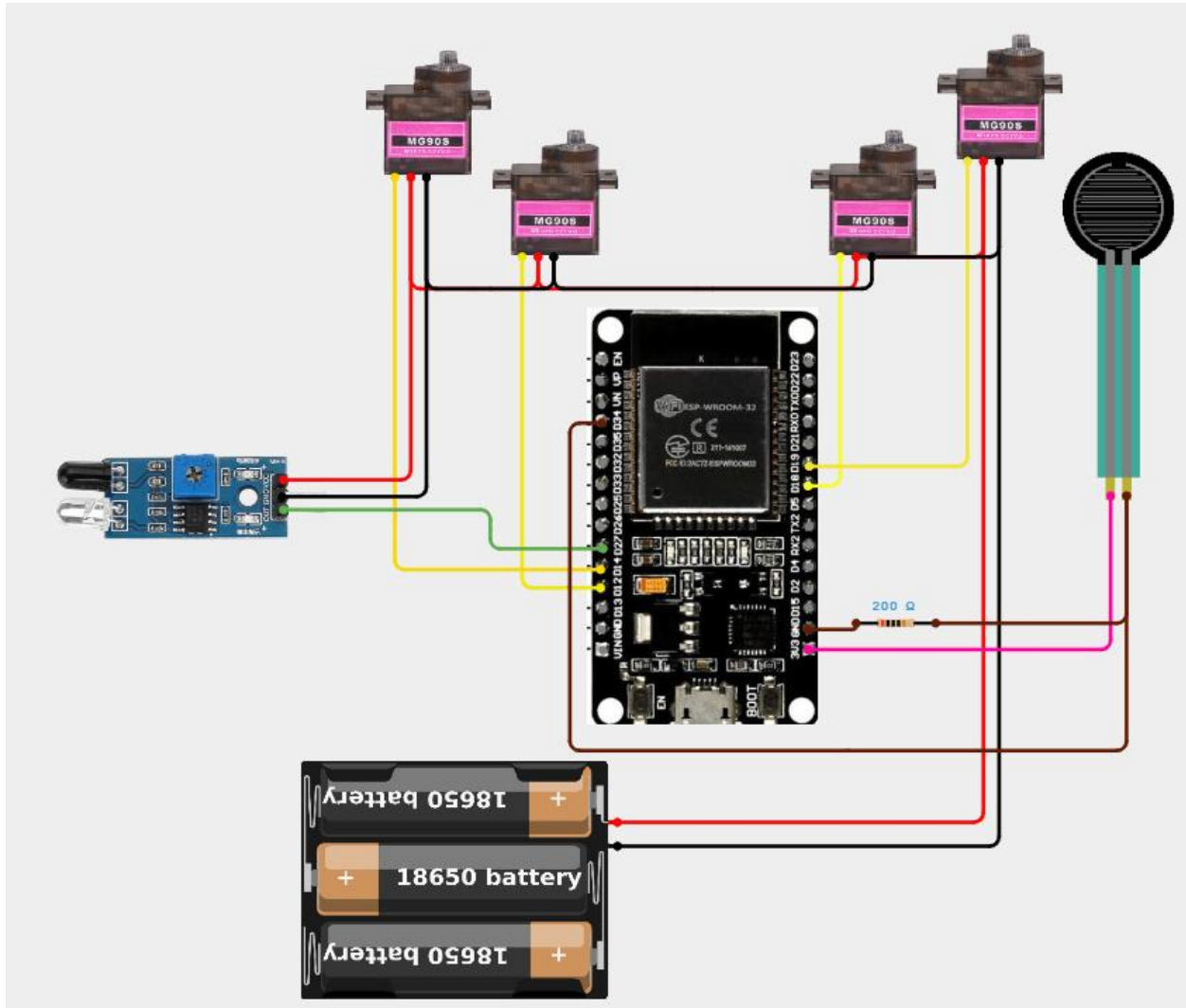


Figure 4 - Circuit Diagram

3.2.4. Hardware and Software Requirements with Justification

- **ESP32 Microcontroller:** Provides sufficient processing capability and wireless support.
- **Servo Motors:** Enable precise angular positioning of arm joints.
- **IR Sensor:** Provides reliable and low-cost object detection.
- **FSR Sensor:** Enables adaptive gripping force to prevent object damage.

3.2.5. Risk Mitigation Plan with Alternative Solution Identification

Risk	Impact	Mitigation Strategy	Alternative Solution
Servo overload	Mechanical failure	Limit payload weight	High-torque servos
Grip slippage	Dropped objects	Tune FSR thresholds	Mechanical spring grip
Power instability	System reset	Separate motor power	Higher-rated adapter
Sensor noise	False detection	Software filtering	Different sensor type

3.2.6. Cost Benefit Analysis

The proposed solution has all the features of an industrial robot, but was conceived as a much less expensive alternative to traditional industrial robots. This is made possible by utilizing inexpensive parts, including the ESP32 microcontroller and standard servos in place of specialty motors. Consequently, this solution can provide the same basic functions as an industrial robot at a fraction of the cost to develop and deploy. This makes it a perfect fit for schools where funding is restricted, and advanced robotic systems are not available to students for use in their studies. The versatility of the system makes it ideal for use in low-volume manufacturing, training centers, laboratories, etc. The design also allows users to customise or upgrade their robotic solution as their needs change, which enhances practicality as well as the long-term viability of the product.

3.3 Performance Requirements

- Object detection response time less than 1 second
- Servo positioning accuracy within acceptable tolerance
- Stable continuous operation under normal workload

3.4 Design Constraints

- Fixed number of servo motors
- Limited power and space availability
- Embedded processing limitations

3.5 Attributes of Software Systems

3.5.1 Reliability

Software systems are dependable when they can perform the same repetitive pick and place tasks repeatedly and reliably under normal operating conditions. Smooth and accurate arm movements are made possible by a dependable servo motor control spring, and sensor technology gives the user a dependable set of readings for the location and weight of the object they will be using, allowing for an accurate amount of power to be applied to those objects, while ensuring that the user's grip remains secure and won't slip free while using the system. Calibration processes will allow for a consistent and repeatable performance; the end user will be able to calibrate the device once, which will enable the system to operate consistently and accurately over multiple sessions.

3.5.2 Availability

The system is fully operational as soon as the user has turned it on and the server has successfully initialized it. The ESP32 will automatically initialize all hardware associated with the robotic arm (including servo motors) when the user turns on the system. This way the user can begin using the robotic arm immediately. Additionally, by automating the initialization process for the user, the likelihood of the user having issues with setup and configuration is greatly reduced, and this allows the user to spend more time focusing on the tasks they wish to accomplish, and less time waiting for the robotic arm to be properly configured.

3.5.3 Security

There are no sensitive users' data and critical information handled by the system, so it has very few software security requirements. Physical access control to the robot arm, power supply, and the programming interface is the main mechanism through which security is achieved. Since the system is operated exclusively in controlled environments (e.g., laboratories or classrooms), this

prevents both unauthorized access to the system, accidental usage, or inadvertent software changes.

3.5.4 Maintainability

The software is written using a module-oriented programming model, where each functional area (sensor data acquisition, motor control, and system decision-making logic) is assigned its own module. As a result, the code is easier to read and troubleshoot, and testing is also simpler. In addition to the structured code and documentation making it easier to read and understand for future developers, the modular programming model also allows for additional features to be added with minimal impact on pre-existing functionality. For example, any new sensors or methods for communication can be added with minimal change to the existing codebase.

3.6 Other Requirements

The following will outline the minimum requirements to ensure safe operation of the robotic system.

- Use caution when operating the Robotic System, in order to minimize the risk of personal injury or damage to hardware components.
- Documented calibration practices and procedures must be kept up to date for the Robot's sensors and ensure the continued reliability of the System's servos.
- After any changes to the System's hardware or software, appropriate testing of the System's functionality must be carried out to confirm that the System is functioning properly;
- The System must be used within the manufacturer defined range of Electrical, Mechanical and Environmental conditions, in order to ensure that it can operate continuously over the extended life of the system.

4. References

[1]. “ESP32 Technical Reference Manual.”

https://documentation.espressif.com/esp32_technical_reference_manual_en.pdf

[2]. “Arduino ESP32 Core Documentation”

<https://docs.espressif.com/projects/arduino-esp32/en/latest/>

[3]. “IR Sensor and FSR Sensor Datasheets”

<https://descargas.cetronic.es/KitSensoresArduino.pdf>

5. Supporting Information

5.1 Table of Contents and Index

Provided at the beginning of the document.

5.2 Appendices

Appendix A - Test Cases

Test Case ID	Test Description	Input / Condition	Expected Result	Actual Result	Status
TC-01	System Power-On Test	Power connected	ESP32 initializes successfully	As expected	Pass
TC-02	Servo Calibration Test	Startup calibration routine	Servos move to home positions	As expected	Pass
TC-03	IR Sensor Detection Test	Object placed in range	Object detected	As expected	Pass

TC-04	No Object Detection	No object present	No pick operation triggered	As expected	Pass
TC-05	Gripper Open Test	Pick sequence initiated	Gripper opens	As expected	Pass
TC-06	Gripper Close Test (Fixed Angle)	Object detected	Gripper closes to preset angle	As expected	Pass
TC-07	Pick Operation	Object within weight limit	Object lifted successfully	As expected	Pass
TC-08	Place Operation	Place command executed	Object released correctly	As expected	Pass
TC-09	FSR Sensor Reading Test	Pressure applied to FSR	FSR values displayed on serial monitor	As expected	Pass
TC-10	FSR-Based Grip Control	FSR linked to gripper servo	Gripper adjusts force dynamically	Not implemented	In Progress

Appendix B - Pin Mapping Table

1. Servo Motor Connections

Component	ESP32 GPIO Pin	Signal Type	Power Source	Description
Base Servo	GPIO 13	PWM Output	External 5–6V	Controls base rotation
Shoulder Servo	GPIO 12	PWM Output	External 5–6V	Controls shoulder movement

Elbow Servo	GPIO 18	PWM Output	External 5–6V	Controls elbow movement
Gripper Servo	GPIO 19	PWM Output	External 5–6V	Opens and closes gripper

All servo grounds are connected to a common GND with ESP32.

2. Sensor Connections

IR Sensor

IR Sensor Pin	ESP32 Pin	Description
VCC	3.3V / 5V	Power supply
GND	GND	Common ground
OUT	GPIO 33	Digital input for object detection

FSR Sensor (Currently Standalone Testing)

FSR Connection	ESP32 Pin	Description
FSR Output	GPIO 34	Analog force measurement
Fixed Resistor	GND	Voltage divider configuration
VCC	3.3V	Sensor excitation

3. Power Supply Connections

Component	Power Source	Description
ESP32	USB / 5V Regulator	Logic power supply
Servo Motors	External 5–6V Supply	Prevents voltage drops
Sensors	ESP32 3.3V / 5V	Low power consumption